

basic concepts

basic concepts

{/* 知识类型: 概念介绍 / 操作步骤 */}{/* 适用场景: 建表设计 / 数据组织与管理 */}

本文介绍 Doris 的分区（Partition）与分桶（Bucket）机制，帮助用户合理设计表结构以提升查询性能与数据管理效率。建议新手按章节顺序阅读：第 1-3 节涵盖核心概念与第一个建表示例，第 4-6 节为进阶特性与设计建议，第 7 节为日常运维所需的查看与修改方法。

1. 概述

Doris 采用 **分区（Partition）+ 分桶（Bucket）** 的两层数据划分方式，将一张表的数据有序地组织到集群的各个节点上：

- **分区**：按列值（如时间、地区）将表水平切分为更小的子集，便于查询裁剪与数据生命周期管理；
- **分桶**：在每个分区内进一步将数据均匀打散到多个数据分片（Tablet）中，以充分利用集群并行能力。

图片说明：doris-data-partitioning

数据流向可概括为：

Table —► Partition —► Bucket —► Tablet（数据分片，存储在 BE 节点上）

合理的分区分桶设计能同时带来：**查询更快**（分区裁剪、并行扫描）、**管理更灵活**（按时归档/清理）、**写入更均匀**（避免热点）。

2. 核心概念

2.1 表结构：行与列

在 Doris 中，数据以表（Table）的形式进行逻辑描述。一张表由行（Row）和列（Column）组成：

- 概念：Row；说明：用户的一行数据
- 概念：Column；说明：描述一行数据中的不同字段

Column 可以进一步分为两大类：

- **列类型：Key 列**；业务对应：维度列；说明：建表语句中 unique key、aggregate key 或 duplicate key 关键字后指定的列
- **列类型：Value 列**；业务对应：指标列；说明：Key 列以外的其他列，聚合方式由建表时指定

在聚合模型中，Key 列相同的行会被聚合成一行，Value 列的聚合方式由用户在建表时指定。关于聚合模型的更多介绍，请参阅 Doris 数据模型。

2.2 分区 (Partition)

分区是指根据表中特定列的值，将数据划分为更小的、互不相交的子集。每一行数据属于且仅属于一个特定的分区，分区是最小的逻辑管理单元。

Doris 支持两种**分区类型**：

- **Range 分区**：按数值或时间范围划分，常用于时间序列数据；
- **List 分区**：按枚举值集合划分，常用于按地区、品类等离散维度切分。

建表时如果不指定分区，Doris 会生成一个对用户透明的默认分区，包含表中所有数据。

合理分区可以带来以下收益：

- **查询性能提升**：通过分区裁剪，系统可以根据查询条件过滤掉不相关的分区，减少数据扫描量，显著降低 I/O 负担，特别适合大规模数据集；
- **管理灵活性**：可按时间、地域等逻辑维度对数据进行分割，便于归档、清理和备份。例如按时间分区可高效管理历史数据与新增数据，支持基于时间的数据维护策略。

2.3 分桶 (Bucket)

分桶是指将一个分区中的数据，按照某种规则进一步划分为更小的、互不相交的数据单元。每一行数据属于且仅属于一个特定的分桶。

与按列值范围划分的分区不同，分桶的目标是将数据**均匀分布**到预定义的桶中，从而减少数据倾斜，并通过提高数据局部性来提升查询执行性能。

Doris 支持两种**分桶方式**：

- **Hash 分桶**：通过计算分桶列值的 crc32 哈希值，并对桶数取模，将数据均匀分布；

- **Random 分桶**：随机分配数据到桶中。使用 Random 分桶时，可结合 `load_to_single_tablet` 参数优化小规模数据的快速写入。

合理分桶可以带来以下收益：

- **数据均匀分布**：减少数据集中或倾斜的风险，避免部分节点或存储设备资源过载；
- **减少热点**：避免某些节点或分区过度负载，提升系统稳定性和处理能力；
- **提高并发性能**：当多个查询请求需要访问同一分区中的不同数据时，分桶可使系统有效地并行处理多个请求，从而提升吞吐量。

2.4 数据分片（Tablet）与节点架构

一个分桶在物理上对应一个**数据分片（Tablet）**，Tablet 是 Doris 中数据管理的最小单元，也是数据移动、复制等操作的基本物理单位。

Doris 集群由两类节点组成：

- **FE 节点（Frontend）**：管理集群元数据（如表、分片信息），负责 SQL 的解析与执行规划；
- **BE 节点（Backend）**：存储 Tablet 数据，负责计算任务的执行；BE 的结果汇总后由 FE 返回给用户。

数据写入与查询时，分区和分桶的协作过程如下：

- **数据写入**：先按分区策略将数据行分配到对应分区，再按分桶策略映射到分区内的具体 Tablet；
- **查询执行**：FE 优化器会根据查询条件裁剪掉不相关的分区与分桶，最大化减少扫描范围。涉及 JOIN 或聚合查询时，可能发生跨节点的 Shuffle，合理的分区分桶设计（必要时配合 Colocate）可显著降低 Shuffle 成本。

3. 第一个建表示例

Doris 的建表是同步命令，SQL 执行完成即返回结果，命令返回成功即表示建表成功。完整建表语法可参考 CREATE TABLE，也可以通过 HELP CREATE TABLE 查看更多帮助信息。

下面是一个采用 Range 分区 + Hash 分桶的最小建表示例：

```
-- Range Partition
CREATE TABLE example_range_tbl
(
  `user_id` LARGEINT NOT NULL COMMENT "用户 id",
```

```

`date` DATE NOT NULL COMMENT "数据灌入日期时间",
`timestamp` DATETIME NOT NULL COMMENT "数据灌入的时间戳",
`city` VARCHAR(20) COMMENT "用户所在城市",
`age` SMALLINT COMMENT "用户年龄",
`sex` TINYINT COMMENT "用户性别",
`last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "用户
最后一次访问时间",
`cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
`max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
`min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
)
PARTITION BY RANGE(`date`)
(
  PARTITION `p201701` VALUES LESS THAN ("2017-02-01"),
  PARTITION `p201702` VALUES LESS THAN ("2017-03-01"),
  PARTITION `p201703` VALUES LESS THAN ("2017-04-01"),
  PARTITION `p2018` VALUES ("2018-01-01"), ("2019-01-01"))
)
DISTRIBUTED BY HASH(`user_id`) BUCKETS 16
PROPERTIES
(
  "replication_num" = "1"
);

```

结合本例理解写入与查询过程：

- 写入时，每行数据先按 date 落入对应的 Range 分区（如 2017-02-15 进入 p201702），再按 user_id 的 Hash 值映射到该分区下 16 个 Tablet 中的某一个；
- 查询 WHERE date = '2017-02-15' 时，优化器只扫描 p201702 分区；若进一步带上 user_id = 123，则仅命中其中一个 Tablet。

4. 进阶：分区模式

除了建表时手动声明分区外，Doris 还支持按时间调度自动创建分区（动态分区）和按写入数据按需创建分区（自动分区）。三种模式对比如下：

- 分区模式：手动分区；何时创建分区：建表时显式声明，或通过 ALTER 添加；适用场景：分区集合稳定、需精细控制
- 分区模式：动态分区；何时创建分区：系统按时间调度规则自动创建/回收；适用场景：时间序列数据，希望自动滚动维护近 N 天/周/月分区

- 分区模式：自动分区；何时创建分区：数据写入时按需创建；适用场景：分区取值不可预知（如多租户、稀疏时间），希望避免预创建

下面给出常见组合的建表示例：

[自动分区 支持在数据导入时根据用户定义的规则自动创建对应分区，使用更为便捷。将基础示例改写为自动 Range 分区：

```
CREATE TABLE example_range_tbl
(
  `user_id` LARGEINT NOT NULL COMMENT "用户 id",
  `date` DATE NOT NULL COMMENT "数据灌入日期时间",
  `timestamp` DATETIME NOT NULL COMMENT "数据灌入的时间戳",
  `city` VARCHAR(20) COMMENT "用户所在城市",
  `age` SMALLINT COMMENT "用户年龄",
  `sex` TINYINT COMMENT "用户性别",
  `last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "用户最后一次访问时间",
  `cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
  `max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
  `min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
)
AUTO PARTITION BY RANGE(date_trunc(`date`, 'month')) --- 使用月作为分区粒度
()
DISTRIBUTED BY HASH(`user_id`) BUCKETS 16
PROPERTIES
(
  "replication_num" = "1"
);
```

如上建表，当数据导入时，Doris 将自动按月级别为 date 列创建对应分区。例如 2018-12-01 与 2018-12-31 会落入同一个分区，而 2018-11-12 会落入另一个分区。自动分区还支持 List 分区，更多用法请查看自动分区文档。

动态分区 是根据现实时间进行自动分区创建与回收的管理方式。将基础示例改写为动态分区：

```
CREATE TABLE example_range_tbl
(
  `user_id` LARGEINT NOT NULL COMMENT "用户 id",
  `date` DATE NOT NULL COMMENT "数据灌入日期时间",
  `timestamp` DATETIME NOT NULL COMMENT "数据灌入的时间戳",
```

```

    `city` VARCHAR(20) COMMENT "用户所在城市",
    `age` SMALLINT COMMENT "用户年龄",
    `sex` TINYINT COMMENT "用户性别",
    `last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "用户
最后一次访问时间",
    `cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
    `max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
    `min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
)
PARTITION BY RANGE(`date`)
()
DISTRIBUTED BY HASH(`user_id`) BUCKETS 16
PROPERTIES
(
    "replication_num" = "1",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "WEEK", --- 分区粒度为周
    "dynamic_partition.start" = "-2", --- 向前保留两周
    "dynamic_partition.end" = "2", --- 提前创建后两周
    "dynamic_partition.prefix" = "p",
    "dynamic_partition.buckets" = "8"
);

```

动态分区支持分层存储、自定义副本数等功能，详见动态分区文档。

自动分区与动态分区各有优势，二者结合可实现分区的灵活按需创建与自动回收：

```

CREATE TABLE example_range_tbl
(
    `user_id` LARGEINT NOT NULL COMMENT "用户 id",
    `date` DATE NOT NULL COMMENT "数据灌入日期时间",
    `timestamp` DATETIME NOT NULL COMMENT "数据灌入的时间戳",
    `city` VARCHAR(20) COMMENT "用户所在城市",
    `age` SMALLINT COMMENT "用户年龄",
    `sex` TINYINT COMMENT "用户性别",
    `last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "用户
最后一次访问时间",
    `cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
    `max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
    `min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
)

```

```

AUTO PARTITION BY RANGE(date_trunc(`date`, 'month')) --- 使用月作为分区粒度
()
DISTRIBUTED BY HASH(`user_id`) BUCKETS 16
PROPERTIES
(
  "replication_num" = "1",
  "dynamic_partition.enable" = "true",
  "dynamic_partition.time_unit" = "month", --- 二者粒度必须相同
  "dynamic_partition.start" = "-2", --- 动态分区自动清理超过两周的历史分区
  "dynamic_partition.end" = "0", --- 动态分区不创建未来分区，完全交给自动分区
  "dynamic_partition.prefix" = "p",
  "dynamic_partition.buckets" = "8"
);

```

关于该功能的细节说明，详见 自动分区与动态分区联用。

5. 进阶：分桶进阶

5.1 自动分桶

当用户不确定合理的分桶数时，可以使用自动分桶让 Doris 完成估计，用户仅需提供预估的表数据量：

```

CREATE TABLE IF NOT EXISTS example_range_tbl
(
  `user_id` LARGEINT NOT NULL COMMENT "用户 id",
  `date` DATE NOT NULL COMMENT "数据灌入日期时间",
  `timestamp` DATETIME NOT NULL COMMENT "数据灌入的时间戳",
  `city` VARCHAR(20) COMMENT "用户所在城市",
  `age` SMALLINT COMMENT "用户年龄",
  `sex` TINYINT COMMENT "用户性别",
  `last_visit_date` DATETIME REPLACE DEFAULT "1970-01-01 00:00:00" COMMENT "用户最后一次访问时间",
  `cost` BIGINT SUM DEFAULT "0" COMMENT "用户总消费",
  `max_dwell_time` INT MAX DEFAULT "0" COMMENT "用户最大停留时间",
  `min_dwell_time` INT MIN DEFAULT "99999" COMMENT "用户最小停留时间"
)
PARTITION BY RANGE(`date`)
(
  PARTITION `p201701` VALUES LESS THAN ("2017-02-01"),
  PARTITION `p201702` VALUES LESS THAN ("2017-03-01"),

```

```

PARTITION `p201703` VALUES LESS THAN ("2017-04-01"),
PARTITION `p2018` VALUES [("2018-01-01"), ("2019-01-01"))
)
DISTRIBUTED BY HASH(`user_id`) BUCKETS AUTO
PROPERTIES
(
  "replication_num" = "1",
  "estimate_partition_size" = "2G" --- 用户估计一个分区将有的数据量，不提供则默认为 10G
);

```

需要注意的是，该方式不适用于表数据量特别大的场景。

5.2 Colocate（同分布）

对于需要频繁进行 JOIN 或聚合查询的多张大表，可以启用 **Colocate** 策略，将相同分桶列值的数据放置在同一物理节点上，避免跨节点 Shuffle，从而显著提升查询性能。详细使用方法请参考 Colocate Join 相关文档。

6. 设计建议

合理的分区分桶设计需要兼顾查询性能、数据管理与系统稳定性。以下是一些通用经验：

6.1 设计目标

1. **均匀数据分布**：选择高基数列作为分桶列，避免数据倾斜导致部分节点过载；
2. **优化查询性能**：分区裁剪可大幅减少扫描数据量；合理分桶数提升计算并行度；必要时利用 Colocate 降低 Shuffle 成本；
3. **灵活数据管理**：按时间分区便于冷/热分层（HDD/SSD），并通过删除历史分区释放存储空间。

6.2 经验值与限制

- **元数据规模**：每个 Tablet 的元数据存储在 FE 与 BE 中，需要合理控制规模：
 - 每 1000 万 Tablet，FE 至少需要约 100 GB 内存；
 - 单个 BE 承载的 Tablet 数应小于 2 万。
- **写入吞吐**：

- 单分区分桶数建议 show create table example_range_tbl

+-----

+-----

-----+


```
| Table | Create Table | +-----+
+-----+
example_range_tbl | CREATE TABLE example_range_tbl ( || user_id largeint(40) NOT
NULL COMMENT '用户 id', || date date NOT NULL COMMENT '数据灌入日期时间', ||
timestamp datetime NOT NULL COMMENT '数据灌入的时间戳', || city varchar(20)
NULL COMMENT '用户所在城市', || age smallint(6) NULL COMMENT '用户年龄', ||
sex tinyint(4) NULL COMMENT '用户性别', || last_visit_date datetime REPLACE NULL
DEFAULT "1970-01-01 00:00:00" COMMENT '用户最后一次访问时间', || cost
bigint(20) SUM NULL DEFAULT "0" COMMENT '用户总消费', || max_dwell_time
int(11) MAX NULL DEFAULT "0" COMMENT '用户最大停留时间', || min_dwell_time
int(11) MIN NULL DEFAULT "99999" COMMENT '用户最小停留时间' || ) ||
PARTITION BY RANGE(date) || (PARTITION p201701 VALUES [('0000-01-01'), ('2017-
02-01')), || PARTITION p201702 VALUES [('2017-02-01'), ('2017-03-01')), ||
PARTITION p201703 VALUES [('2017-03-01'), ('2017-04-01')) || DISTRIBUTED BY
HASH(user_id) BUCKETS 16 || PROPERTIES ( || "replication_allocation" =
"tag.location.default: 1", || "is_being_synced" = "false", || "storage_format" =
"V2", || "light_schema_change" = "true", || "disable_auto_compaction" = "false" |
|| ); +-----+
+-----+
```

7.2 通过 SHOW PARTITIONS 查看分区列表

通过 `SHOW PARTITIONS FROM` 可以查看表的分区列表及详细信息：

```
` `` `sql
> show partitions from example_range_tbl
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
| PartitionId | PartitionName | VisibleVersion | VisibleVersionTime | State | PartitionKey |
Range | DistributionKey | Buckets | ReplicationNum |
StorageMedium |
| CooldownTime | RemoteStoragePolicy | LastConsistencyCheckTime | DataSize |
IsInMemory | ReplicaAllocation | IsMutable |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
```

```

+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
| 28731 | p201701 | 1 | 2024-01-25 10:50:51 | NORMAL | date | [types:
[DATEV2]; keys: [0000-01-01]; ..types: [DATEV2]; keys: [2017-02-01]; ) | user_id | 16 | 1
| HDD
| 9999-12-31 23:59:59 | | | 0.000 | false | tag.location.default: 1 | true
|
| 28732 | p201702 | 1 | 2024-01-25 10:50:51 | NORMAL | date | [types:
[DATEV2]; keys: [2017-02-01]; ..types: [DATEV2]; keys: [2017-03-01]; ) | user_id | 16 | 1
| HDD
| 9999-12-31 23:59:59 | | | 0.000 | false | tag.location.default: 1 | true
|
| 28733 | p201703 | 1 | 2024-01-25 10:50:51 | NORMAL | date | [types:
[DATEV2]; keys: [2017-03-01]; ..types: [DATEV2]; keys: [2017-04-01]; ) | user_id | 16 | 1
| HDD
| 9999-12-31 23:59:59 | | | 0.000 | false | tag.location.default: 1 | true
|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+

```

7.3 修改分区

通过 ALTER TABLE ADD PARTITION 可以为表新增分区：

```
ALTER TABLE example_range_tbl ADD PARTITION p201704 VALUES LESS THAN("2020-05-01") DISTRIBUTED BY HASH(`user_id`) BUCKETS 5;
```

更多分区修改操作，请参考 SQL 手册 ALTER-TABLE-PARTITION。

7.4 分区检索

partitions 表函数和 information_schema.partitions 系统表记录了集群的分区信息。在自动管理分区时，可以通过它们提取分区信息。

查找指定值所属的分区

在 Auto Partition 表中查找某个值对应的分区：

```
mysql> select * from partitions("catalog"="internal", "database"="optest",
"table"="DAILY_TRADE_VALUE") where PartitionName = auto_partition_name('range',
'year', '2008-02-03');
```

PartitionId	PartitionName	VisibleVersion	VisibleVersionTime	State	PartitionKey	DistributionKey	Buckets	ReplicationNum	StorageMedium	CooldownTime	RemoteStoragePolicy	LastConsistencyCheckTime	DataSize	IsInMemory	ReplicaAllocation	IsMutable	SyncWithBaseTables	UnsyncTables
127095	p20080101000000	2	2024-11-14 17:29:02	NORMAL	TRADE_DATE	[types: [DATEV2]; keys: [2008-01-01]; ..types: [DATEV2]; keys: [2009-01-01];)	TRADE_DATE	10	1	HDD	9999-12-31 23:59:59	\N	985.000 B	0	tag.location.default: 1	1	1	\N

1 row in set (0.30 sec)

在 List 分区表中查找对应的分区：

```
mysql> select * from information_schema.partitions where TABLE_SCHEMA='optest' and
TABLE_NAME='list_table1' and PARTITION_NAME=auto_partition_name('list', null);
```

TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	PARTITION_NAME	SUBPARTITION_NAME	PARTITION_ORDINAL_POSITION	SUBPARTITION_ORDINAL_POSITION	PARTITION_METHOD	SUBPARTITION_METHOD	PARTITION_EXPRESSION	SUBPARTITION_EXPRESSION	PARTITION_DESCRIPTION	TABLE_ROWS	AVG_ROW_LENGTH	DATA_LENGTH	MAX_DATA_LENGTH	INDEX_LENGTH	DATA_FREE	CREATE_TIME	UPDATE_TIME	CHECK_TIME	CHECKSUM
---------------	--------------	------------	----------------	-------------------	----------------------------	-------------------------------	------------------	---------------------	----------------------	-------------------------	-----------------------	------------	----------------	-------------	-----------------	--------------	-----------	-------------	-------------	------------	----------

PARTITION_COMMENT		NODEGROUP		TABLESPACE_NAME					
internal	optest	list_table1	pX	NULL		0		0	
LIST	NULL	str	NULL	(NULL)		1		1266	
1266	0	0	0	0	2024-11-14 19:58:45	0000-00-00 00:00:00		0	

```

01-01'), ('2013-01-01')) |      1 |      985 |      985 |      0 |      0 |      0 |      0 | 2024-11-
14 17:29:02 | 0000-00-00 00:00:00 |      0 |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
1 row in set (0.65 sec)

```

8. 扩展阅读

- [手动分区
- 动态分区
- 自动分区
- 数据分桶
- 常见问题
- CREATE TABLE 语法
- ALTER TABLE PARTITION